

MemGate: An Adaptive Memory Layer for Long-Running LLM Agents

Simar Singh Khanuja - 70562200088

Yash Ramchandani - 70562300097

SVKM's NMIMS, Indore Campus

Introduction

LLM agents are now used for work that runs over long sessions: coding assistants, research agents, and personal assistants that are expected to remember what happened earlier. To stay coherent, most of them simply append every message to a growing prompt and send the whole history back to the model on each turn. This works early on, but it does not scale. The cost of self-attention rises with the square of the sequence length, and the key-value (KV) cache the model keeps in memory grows with every token added. So the longer an agent runs, the slower and more expensive each reply becomes, until it either fills the context window or gets too costly to continue. Much of that stored history is also redundant, greetings and acknowledgements that no longer matter.

MemGate sits between the agent and the model and manages that history actively, keeping only what matters in full and compressing or dropping the rest.

Problem Statement

As a session grows, the entire history is re-sent to the model on every turn. Because attention scales with the square of the length and the KV cache grows with every token, long-running agents get steadily slower and more expensive, and eventually hit the context limit.

The usual fixes are blunt. A fixed sliding window simply drops the oldest messages, while plain summarization compresses everything the same way and loses the specific names, numbers, and decisions the agent needs later. Neither one decides what is actually worth keeping.

Core problem: agents pay a growing cost-and-latency tax for memory they mostly do not need, and current fixes forget the wrong things.

Objectives

Objective 1: Adaptive compression

The Adaptive Memory Decision Engine scores each part of the conversation by importance, then keeps it in full, rewrites it as a short summary, or drops it, so the working context stays inside a fixed token budget.

Objective 2: Hierarchical memory with recall

Build a tiered store (short-term, working, long-term) that moves information between levels much like an OS moves data between cache, RAM, and disk, with a forgetting rule and a way to bring a dropped memory back when it is relevant again.

Objective 3: Evaluation on long tasks

Test on long, multi-session agent tasks and report the trade-off between tokens saved and answer quality, using measures such as tokens per turn, latency, retrieval accuracy, and task success.

Scope of the Project

Project definition: MemGate is a software layer that wraps an existing LLM (run locally or reached through an API) and controls what enters the prompt on each turn, treating the base model as a black box.

In scope

- A controller that scores, compresses, stores, forgets, and retrieves conversational memory
- A three-tier store: short-term buffer, summarized working memory, and a vector-backed long-term store
- Testing with at least one open model and one hosted API
- Evaluation on long-conversation and multi-session benchmarks

Out of scope (for now)

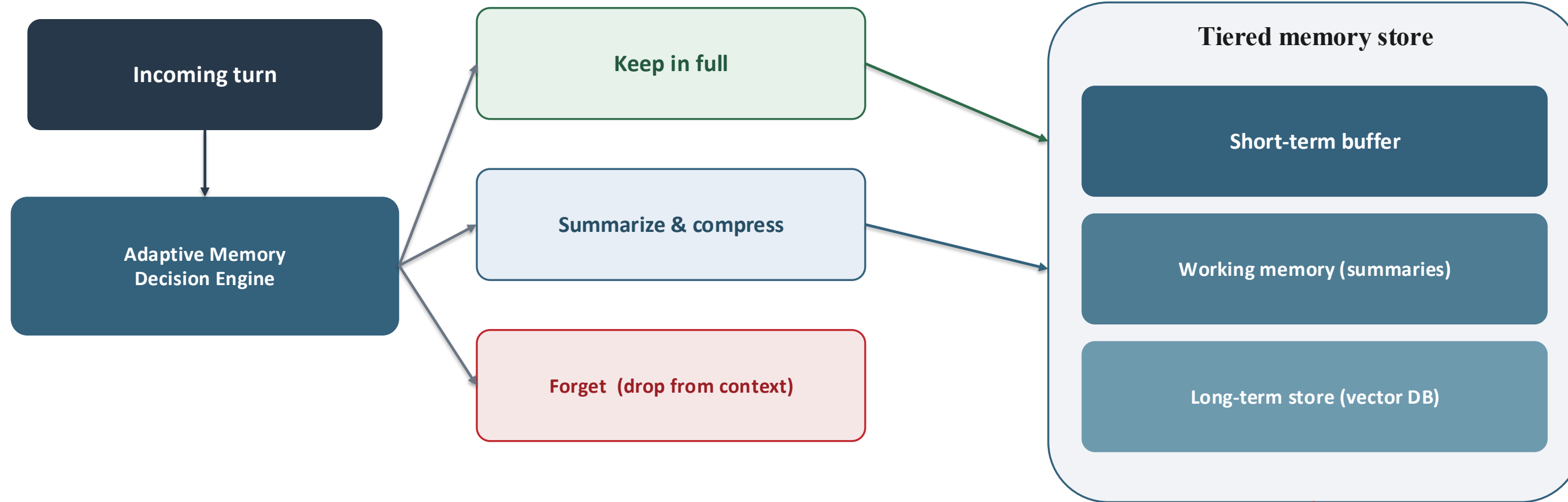
- Re-training or fine-tuning the base LLM itself
- Image or audio memory; this project is text only
- A polished product or user interface — the deliverable is a working prototype plus results

Need of the Project

- **Long chats are expensive:** token pricing plus quadratic attention make extended sessions costly, and cutting stored tokens cuts the bill directly.
- **Context windows still fill up:** even large windows run out over long or multi-day use, so agents need memory that outlives the window.
- **Current fixes are crude:** sliding windows and flat summaries cannot tell old-but-important from recent-but-trivial.
- **Speed matters for real use:** bigger prompts mean slower replies, which hurts interactive agents; a smaller context means faster responses.
- **Useful assistants must remember:** personal and enterprise agents are only worth it if they recall the right past detail cheaply, which needs managed memory.

Proposed System

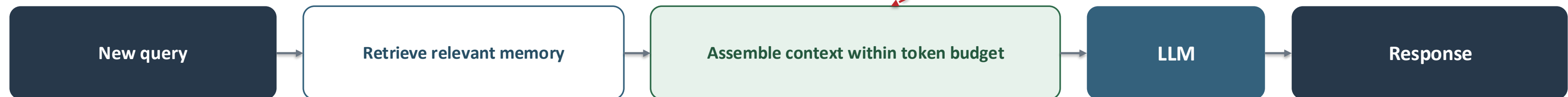
MANAGING MEMORY (write path)



How to read it

Each turn is scored by the decision engine, then kept, summarized, or forgotten. Kept and summarized items live in the three tiers. When a new query needs old information, the retriever pulls it back and the assembler rebuilds a context that fits the budget before the model runs.

USING MEMORY (read path)



System Architecture

Agent / Application Layer — the LLM agent that produces and consumes turns (coding assistant, research agent, chatbot)

MemGate — Memory-Management Layer

Adaptive Memory
Decision Engine
(score & route)

Compressor /
Summarizer

Retriever
(semantic search)

Context Assembler
(token budget)

Memory Store

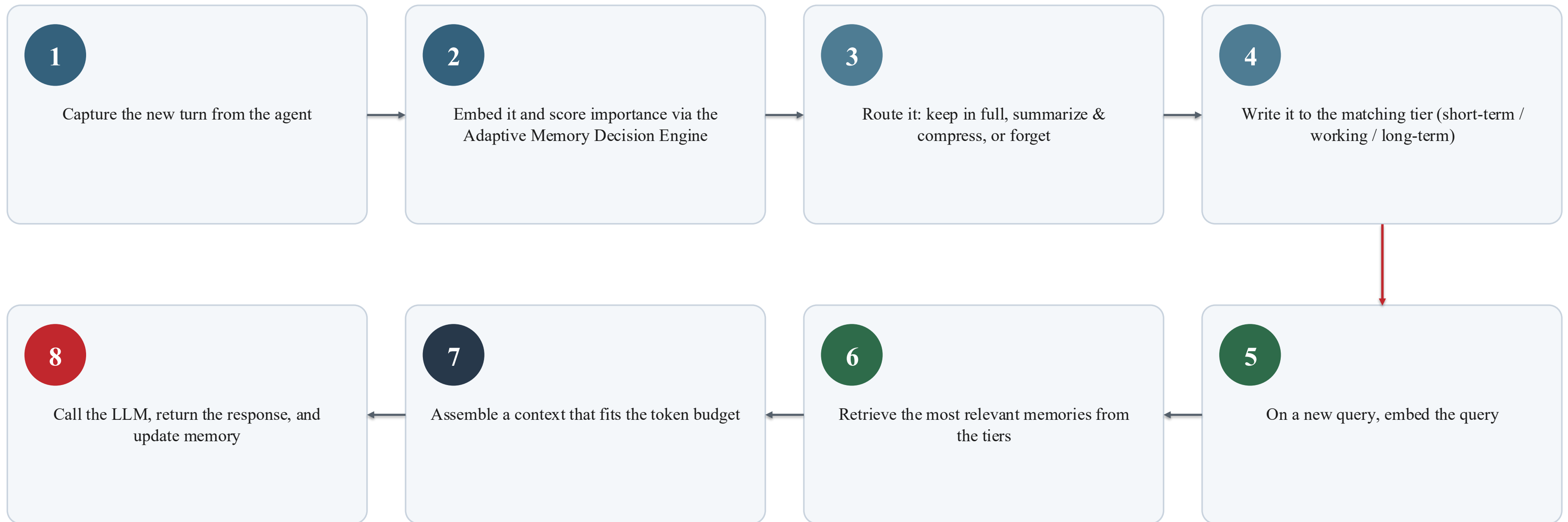
Short-term buffer

Working memory (summaries)

Long-term store (vector DB)

Model Layer — Base LLM (open model or hosted API), treated as a black box

Implementation Workflow



Tools & Technology

- **Core & ML:** Python, PyTorch, NumPy, Pandas
- **Language models:** open models (Llama 3, Mistral) via Hugging Face Transformers; a hosted API (OpenAI / Anthropic) for comparison
- **Agent scaffolding:** LangChain / LlamaIndex as the baseline setup we build on and improve
- **Embeddings & storage:** sentence-transformers for embeddings; FAISS or Chroma as the long-term vector store
- **Compression:** base-LLM summarization plus prompt-compression baselines (LLMLingua-style); tokenizer-level budgeting (tiktoken / HF tokenizers)
- **Workflow & analysis:** Google Colab / university GPU, Git and GitHub, Matplotlib for results

Feasibility of Implementation

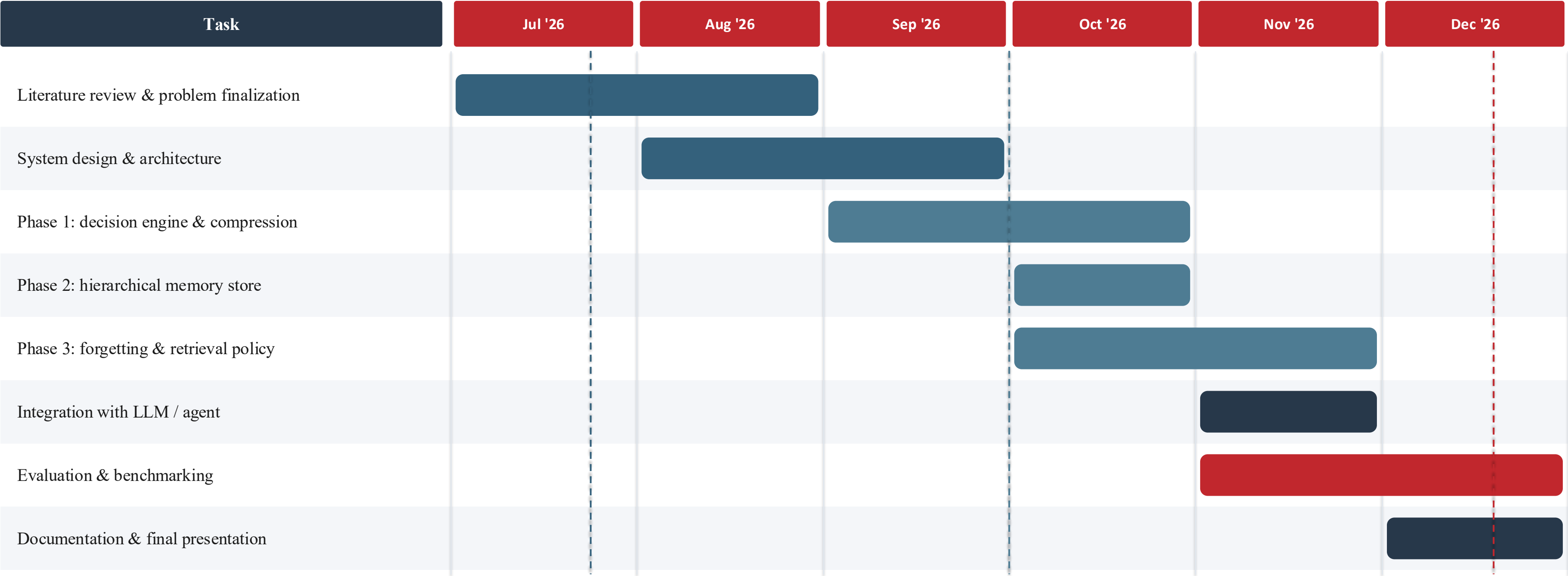
- **Technical:** builds on mature parts — embeddings, vector search, and LLM summarization all exist. The new work is the control policy, not a new model, and it runs on a single GPU or Colab.
- **Data:** public long-conversation benchmarks (LoCoMo, Multi-Session Chat, LongBench) are available, so there is no costly data collection.
- **Cost:** mostly open-source tools; the only real expense is limited API calls, which fits a student budget.

Key constraints and mitigations

- **Limited compute** → smaller open models locally, large model via API
- **Over-forgetting loses detail** → the decision engine plus a retrieval fallback; tune the token budget
- **Measuring recall is hard** → probe questions on standard long-context benchmarks
- **Large system, limited time** → staged phases (compression → hierarchy → retrieval), each demorable

Verdict: feasible within one semester as a staged research prototype.

Gantt Chart





Literature Review (1 of 2)

Paper Title	Dataset / Benchmark	Technology Used	Achievements	Limitations	Gap Addressed (Proposed Work)
MemGPT — Packer et al. (2023)	Multi-Session Chat; document QA	Tiered memory; LLM self-editing via function calls	Pages context in and out to work beyond the window ("LLM as an OS")	Leaves memory decisions to the LLM; no adaptive importance scoring or compression	Adds an adaptive importance scoring policy and compression on top of tiered storage
MemoryBank — Zhong et al. (2024)	Long-term dialogue (SiliconFriend)	Vector memory + Ebbinghaus forgetting curve	Time-decayed forgetting and reinforcement for human-like recall	Forgets by elapsed time, not importance; no budget compression	Forgets by adaptive importance scoring and compresses to a fixed token budget
Generative Agents — Park et al. (2023)	Sandbox agent simulation	Memory stream + retrieval (recency, importance, relevance) + reflection	Retrieval scoring and reflection sustain coherent long-horizon behaviour	Stores the full stream (unbounded); not built to cut tokens or cost	Bounds the store via compression and forgetting; keeps retrieval scoring
StreamingLLM — Xiao et al. (2024)	Long-stream language modelling (up to millions of tokens)	Attention sinks + rolling KV cache	Stable generation over very long streams, up to ~22x faster	Keeps only a window plus sinks; old content is dropped for good	Moves old content to compressed and long-term tiers so it can return
H2O — Zhang et al. (2023)	OPT / LLaMA across tasks	KV-cache eviction keeping "heavy-hitter" tokens	Large cache savings and higher throughput with little quality loss	Token / KV level within one context; no semantic long-term memory	Manages memory at the conversation level and adds long-term recall

Evaluation will use the following metrics: tokens per turn, cost and latency reduction, memory-retrieval accuracy, and end-task success (F1 / exact-match) on long-conversation benchmarks.



Literature Review (2 of 2)

Paper Title	Dataset / Benchmark	Technology Used	Achievements	Limitations	Gap Addressed (Proposed Work)
LLMLingua — Jiang et al. (2023)	GSM8K, BBH, and others	Budget controller + token-level prompt compression	Up to ~20x prompt compression with small quality loss	Compresses a given prompt; does not decide what to store or forget	Uses compression as one action inside a keep / forget / retrieve policy
Gist Tokens — Mu et al. (2023)	Instruction-following (LLaMA-7B, FLAN-T5)	Distils a prompt into a few reusable "gist" vectors	Up to ~26x compression, cacheable and reusable	Aimed at fixed instructions, not evolving dialogue memory	Applies learned compression to changing history and manages its lifecycle
AutoCompressors — Chevalier et al. (2023)	Long-context LM; in-context learning	Recursive "summary vectors" used as soft prompts	Compresses long context into compact vectors; cuts inference cost	Vectors are opaque; no explicit forgetting or fact retrieval	Keeps readable summaries plus a retrievable long-term store
Agent Memory Survey — Zhang et al. (2024)	— (survey)	Taxonomy of agent memory (forms, operations, evaluation)	Organizes the field and motivates why memory matters for agents	Survey only; no system or implementation	Instantiates several of its recommendations in one working system
A-MEM — Xu et al. (2025)	LoCoMo multi-hop QA (six foundation models)	Zettelkasten-style notes; dynamic linking and revision	2–6x better multi-hop reasoning with token savings	Focuses on linking notes; less on strict budget compression	Adds budget-driven compression and forgetting to self-organizing memory

Evaluation will use the following metrics: tokens per turn, cost and latency reduction, memory-retrieval accuracy, and end-task success (F1 / exact-match) on long-conversation benchmarks.

References

- Packer, Charles, et al. "MemGPT: Towards LLMs as Operating Systems." arXiv preprint arXiv:2310.08560 (2023).
- Zhong, Wanjun, et al. "MemoryBank: Enhancing Large Language Models with Long-Term Memory." Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, no. 17, 2024, pp. 19724–19731.
- Park, Joon Sung, et al. "Generative Agents: Interactive Simulacra of Human Behavior." Proc. 36th Annual ACM Symposium on User Interface Software and Technology (UIST), 2023.
- Xiao, Guangxuan, et al. "Efficient Streaming Language Models with Attention Sinks." International Conference on Learning Representations (ICLR), 2024.
- Zhang, Zhenyu, et al. "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models." Advances in Neural Information Processing Systems (NeurIPS), 2023.
- Jiang, Huiqiang, et al. "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models." Proc. 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023, pp. 13358–13376.
- Mu, Jesse, Xiang Lisa Li, and Noah Goodman. "Learning to Compress Prompts with Gist Tokens." Advances in Neural Information Processing Systems (NeurIPS), 2023.
- Chevalier, Alexis, et al. "Adapting Language Models to Compress Contexts." Proc. 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023.
- Zhang, Zeyu, et al. "A Survey on the Memory Mechanism of Large Language Model-Based Agents." arXiv preprint arXiv:2404.13501 (2024).
- Xu, Wujiang, et al. "A-MEM: Agentic Memory for LLM Agents." Advances in Neural Information Processing Systems (NeurIPS), 2025. arXiv:2502.12110.

Thank You

Backup — Implementation Roadmap

(Backup slide — shown on request; not part of the main flow.)

Phase	Timeline	Focus	Key deliverable
Phase 0	Jul – Aug	Literature review, problem finalization, choose datasets (LoCoMo, MSC, LongBench)	Finalized scope and baseline setup
Phase 1	Sep – Oct	Adaptive Memory Decision Engine + compression (keep / summarize / forget)	Scoring and compression on one open model
Phase 2	Oct	Hierarchical store (short / working / long-term) with FAISS or Chroma	Tiered memory with writes
Phase 3	Oct – Nov	Forgetting policy, retriever, and context assembler (token budget)	Full read path with recall
Phase 4	Nov	Integrate open model + hosted API; baselines (sliding window, full history, LLMLingua)	End-to-end agent with baselines
Phase 5	Nov – Dec	Evaluation, ablations, and write-up	Results, report, and final presentation